BBVA
API Market

Technical Documentation

BBVA, best bank with corporate
financial solutions via API in 2024

HTTP signature

Authentication

HTTP signature introduction

HTTP signature

BBVA combines OAuth 2.0 with digital signature. Those methods guarantee that the Third Party is properly authenticated as the integrity in the communications is secured.

Each API request must be signed using the private key agreed between Third Party and BBVA.

*For the Private key there are two options:
1. BBVA generates a Private key for each APP, by including it in a .p8 file.
2. Third Party uses it's own Private key and gives BBVA the public key.

The digital signatures in the API calls must use the signing HTTP messages IETFG draft document definition.

Example of how to generate the signature of a petition using **OpenSSL**.

BBVA provides a .p8 certificate. This file is distributed together with the rest of the credentials required for the BBVA connection.

Using as **example** the Validations API, the endpoint is:

```
post 'https://apis.es.bbvaapimarket.com/es/validations/v1/account-check
```

1

public key

To obtain the public key associated with the private key, the following command can be executed:

```
openssl rsa -in key.p8 -pubout -out rsa.pub
```

It will generate the **rsa.pub** file with the public key of the private key.
The public key will be used for the correct verification of the signature.

2

Create the signing string: toSign.txt

To generate the signature we will have two files: the contents of the string to be signed called **toSign.txt** and a json file to send on the request called **body.json**.

toSign.txt

The signature string format is:

```
“(request-target):${method} ${path}\nhost:${host}\ndate:${date}\ndigest:${digest}”
```

Where "\n" is the equivalent of a line break.
Considering there is a space between ${method} and ${path}.

(request_target):

${method} It is the verb used in the call. (Ex. POST, GET, PUT...).

${path} is the request path. (In our example: /es/validations/v1/account-check)

host:

${host} is the host that will receive the request (Ex. apis.es.bbvaapimarket.com)
Please, consult **host domain definitions**** to understand the correct configuration on your systems.

date:

${date} is the value of the date at the time the request is made, using **date format***.

digest: ${digest}

BBVA uses the RFC 3230 standard accepting the hash algorithms SHA-256 and SHA-512. So we will perform:
1. Body coded in **UTF-8**
2. Hash of the previous body in **SHA-256 o SHA-512**
3. Hash result in **BASE64**

Note that in cases where the body is empty, the hash algorithm must be applied to the empty string.

There are several online tools available to calculate the hash value of a body (GitHub SHA256, GitHub SHA512). Note that the absence of a body in an API request is considered an empty string for summary calculation purposes, and that everything in the body is significant, including square brackets, spaces or line breaks.

Example of toSign.txt:

```
(request-target):post /es/validations/v1/account-check
host:sbx.es.bbvaapimarket.com
date:Mon, 8 Apr 2024 13:20:03 GMT
digest:SHA-512=Fqi4rr1sWWeMrUAgiHnncbjqmmPmCt2DWW67+/0InuWPKRfi0DFnonkz7Ok2J29ezp9iur6eHO
MXUxU2R+4BbA==
```

body.json

The contents of the file body.json is the body of the call we will make (request_body). In this case we will take one of the recordings included in the testing plan of Validations API, that gives a **code : confirmed**.

Example of body.json

```
{ "reference": "TIC018201282023092801005040610571880000000000000000000000000000000000000" }
```

We generate the digest of the example:

```
cat body.json | openssl dgst -sha512 -binary | base64 -w0
```

```
Fqi4rr1sWWeMrUAgiHnncbjqmmPmCt2DWW67+/0InuWPKRfi0DFnonkz7Ok2J29ezp9iur6eHOMXUxU2R+4BbA==
```

Important:

The line break format must be Unix (LF), and no Windows (CR LF) or Mac (CR)

The last line must not have line break, can be checked by 'cat' the file. If the prompt follows the line, it is correct, if there is a line break between the last line of the file and the prompt, the last line break is left over.

To make sure that the last line break is not included, the file can be generated as follows:

Example to obtain toSign.txt:

```
echo -ne "(request-target):post /es/validations/v1/account-check/
nhost:sbx.es.bbvaapimarket.com/ndate:Mon, 8 Apr 2024 13:20:03 GMT/
ndigest:SHA-512=Fqi4rr1sWWeMrUAgiHnncbjqmmPmCt2DWW67+/0InuWPKRfi0DFnonkz7Ok2J29ezp9iur6eH
OMXUxU2R+4BbA==" > toSign.txt
```

3

Sign the toSign.txt content with the private key

Once we have the file with the contents to be signed, we sign it with the private key, it generates the sign binary in the sign.text.sha512 file with the following command:

```
openssl dgst -sha512 -sign key.p8 -out sign.text.sha512 toSign.txt
```

Once signed we pass the contents of the file of our example to base64.

Example:

```
cat sign.text.sha512 | base64 -w0
```

```
Uxzm0I8ImtQtlUN0jYpMOvpBsd9UPgBYSgXTqvTZcsgGBzcYe/0jpWM/9hmYOv/S70JwfVAbw7jwwrKaC6FPl095y
4UbX3pHJHXK61pp6uoKe77zi4apb4wsqOKWSR2Tr4zHo4NyOVv6hSEPI9AhEpbRAi7SnNW1JIkmHczw5aFH9xYzuv
Cx2A/SrhHFXjZixkjVgjIFjDghM2qzuFJU3dVFD6nPzrNoRf6YK3W3wLTtB/NQeOssyGR9R4Rtd2EXUiXTonvYQ/s
TsGNxqIf+nU7MdpowrjO/YAJGKSEqmmjRIO6uPyJVdQkimyHwJ500ynyUbSaFaQy8JoeWZjk14tuz4kqFa2ptlnPi
k1pirupm2gyXIMpKrZcuoco47W6YXEqy6nORWYL3TLunjeq7/so5XLItI7MucboP4PRKwG5mLhtSqXY5JMB7aknN9
c3XeYhB1BJ2t8EuTFL/f6m8pIEwwjgTLowIwSy7J0gQdDcRpxSDBwLfOeOvTpnTbkmcj6ehejdi3C979aOVAOGUKX
YedYhrh0JMiW+5F2LO14E+SSe2sygDLyI42FEffrdKsbbXHiGaWr5GEOSzbiBE2KVxBToaUUhPStAOOJ1yMaqSFrb
dsm9/0ed4OpFrVobryxN6aWrDen9WnKgBybmX3kTOqvC2cRgrp+QbYLXtpgA=
```

We can verify the generated signature and the public key with the following command:

```
openssl dgst -sha512 -verify rsa.pub -signature sign.text.sha512 toSign.txt
```

```
Verified OK
```

Considerations:

- **Be careful with spaces, there should be no spaces between values.**
- **Depends on the content to be encoded in base64, sometimes the last character is one or two "=", should not be removed.**
- **The date included in the signature and the date sent in the request (Header date) must be exactly the same.**
- **It is possible to add more headers in the signature by including them in the "headers" parameter of the signature and in the content to be signed.**

*Date format

In accordance to standard rfc1123 : "EEE, dd MMM yyyy HH:mm:ss Z", always in English. Where:
EEE = "Mon" /"Tue"/ "Wed" /"Thu"/ "Fri" /"Sat"/ "Sun"
MMM = "Jan" / "Feb" / "Mar" / "Apr"/ "May" / "Jun"/ "Jul" / "Aug"/ "Sep" / "Oct"/ "Nov" /"Dec"
Z= "UT" / "GMT"/ "EST" / "EDT"/ "CST" / "CDT"/ "MST" / "MDT"/ "PST" / "PDT"/ 1ALPHA/ (("+" / "-") 4DIGIT)

Example: "Mon, 08 Apr 2024 13:20:03 GMT"

**Host domain definition

For SANDBOX and LIVE the domain will be:

https://sbx.COUNTRYCODE. bbvaapimarket.com
https://apis.COUNTRYCODE. bbvaapimarket.com

Where the COUNTRYCODE is:

COUNTRY	COUNTRYCODE	EXAMPLE
Argentina	ar	https://apis.ar.bbvaapimarket.com
Colombia	co	https://apis.co.bbvaapimarket.com
España	es	https://apis.es.bbvaapimarket.com
Mexico	mx	https://apis.mx.bbvaapimarket.com
Perú	pe	https://apis.pe.bbvaapimarket.com

Only the PIVOT global APIs shall not carry a COUNTRY CODE:

https://sbx. bbvaapimarket.com
https://apis. bbvaapimarket.com

Do not hesitate to consult your BBVA Technical Account Manager if you do not know your API host.